



GENERICOS <T>

¿QUÉ ES UN GENÉRICO?

Los genéricos nos permiten definir clases, métodos y estructuras de datos que pueden operar con cualquier tipo de datos, mientras mantienen la seguridad de tipos. En lugar de especificar un tipo de datos concreto, utilizamos un parámetro de tipo, comúnmente representado por "T", que puede ser sustituido por cualquier tipo.

.

SUS VENTAJAS

- **Reutilización de código:** Puedes usar la misma clase o método con diferentes tipos de datos.
- **Seguridad de tipos:** Los errores de tipo se detectan en tiempo de compilación en lugar de en tiempo de ejecución.
- **Rendimiento:** Al evitar conversiones de tipos innecesarias, se mejora el rendimiento del programa.

SINTAXIS

```
public class ClaseGenerica<T> : MonoBehaviour
{
    private List<T> items = new List<T>();

    public void AddItem(T item)
    {
        items.Add(item);
    }

    public void RemoveItem(T item)
    {
        items.Remove(item);
    }

    public List<T> GetItems()
    {
        return new List<T>(items);
    }
}
```

- **public class GenericManager<T>:**
Declara una clase pública llamada GenericManager que es genérica, lo que significa que puede manejar cualquier tipo T.
- **: MonoBehaviour:** Indica que GenericManager<T> hereda de MonoBehaviour, lo que permite que esta clase se adjunte a objetos en una escena de Unity.

SINTAXIS

```
public class ClaseGenerica<T> : MonoBehaviour
{
    private List<T> items = new List<T>();

    public void AddItem(T item)
    {
        items.Add(item);
    }

    public void RemoveItem(T item)
    {
        items.Remove(item);
    }

    public List<T> GetItems()
    {
        return new List<T>(items);
    }
}
```

private List<T> items: Declara un campo privado items que es una lista de elementos de tipo T. Esta lista se inicializa como una nueva lista vacía. items se usa para almacenar instancias de T.

SINTAXIS

```
public class ClaseGenerica<T> : MonoBehaviour
{
    private List<T> items = new List<T>();

    public void AddItem(T item)
    {
        items.Add(item);
    }

    public void RemoveItem(T item)
    {
        items.Remove(item);
    }

    public List<T> GetItems()
    {
        return new List<T>(items);
    }
}
```

- **public void AddItem(T item):**

Declara un método público AddItem que acepta un parámetro de tipo T llamado item.

- **items.Add(item):** Añade el item proporcionado a la lista items.

SINTAXIS

```
public class ClaseGenerica<T> : MonoBehaviour
{
    private List<T> items = new List<T>();

    public void AddItem(T item)
    {
        items.Add(item);
    }

    public void RemoveItem(T item)
    {
        items.Remove(item);
    }

    public List<T> GetItems()
    {
        return new List<T>(items);
    }
}
```

public void RemoveItem(T item):

Declara un método público RemoveItem que acepta un parámetro de tipo T llamado item.

- **items.Remove(item):** Elimina el item proporcionado de la lista items

SINTAXIS

```
public class ClaseGenerica<T> : MonoBehaviour
{
    private List<T> items = new List<T>();

    public void AddItem(T item)
    {
        items.Add(item);
    }

    public void RemoveItem(T item)
    {
        items.Remove(item);
    }

    public List<T> GetItems()
    {
        return new List<T>(items);
    }
}
```

- **public List<T> GetItems():**
Declara un método público GetItems que no acepta parámetros y devuelve una lista de elementos de tipo T.
- **return new List<T>(items):**
Crea y devuelve una nueva lista que es una copia de la lista items. Esto se hace para evitar que el llamador del método modifique directamente la lista interna items.

CASO DE USO

```
1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4
5  public class Item : MonoBehaviour
6  {
7      public string itemName;
8
9      void Start()
10     {
11         Debug.Log("Item " + itemName + " has spawned.");
12     }
13 }
14
```

```
1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4
5  public class Enemy : MonoBehaviour
6  {
7      public string enemyName;
8
9      void Start()
10     {
11         Debug.Log("Enemy " + enemyName + " has spawned.");
12     }
13 }
14
15
16
```

Supongamos que queremos crear dos listas distintas utilizando la clase “ClaseGenerica”. Para eso debemos crear dos clases en donde podemos realizar la implementación. En este ejemplo se crearon la clase item y la clase enemy.

CASO DE USO-SIN GENERICOS

```
public class EnemyInventory
{
    private List<Enemy> enemies = new List<Enemy>();

    public void AddEnemy(Enemy enemy)
    {
        enemies.Add(enemy);
    }

    public void RemoveEnemy(Enemy enemy)
    {
        enemies.Remove(enemy);
    }

    public Enemy GetWeapon(string enemyName)
    {
        return enemies.Find(w => w.enemyName == enemyName);
    }
}
```

Deberíamos crear una clase por cada lista que querramos manipular.

```
public class PotionInventory
{
    private List<Item> potions = new List<Item>();

    public void AddPotion(Item potion)
    {
        potions.Add(potion);
    }

    public void RemovePotion(Item potion)
    {
        potions.Remove(potion);
    }

    public Item GetPotion(string potionName)
    {
        return potions.Find(p => p.itemName == potionName);
    }
}
```

CASO DE USO-CON GENERICOS

```
public class GameController : MonoBehaviour
{
    private ClaseGenerica<Enemy> enemyList;
    private ClaseGenerica<Item> itemList;

    void Start()
    {
        enemyList = new ClaseGenerica<Enemy>();
        itemList = new ClaseGenerica<Item>();

        Enemy enemy1 = new GameObject("Enemy1").AddComponent<Enemy>();
        enemy1.enemyName = "Goblin";
        enemyList.AddItem(enemy1);

        Enemy enemy2 = new GameObject("Enemy2").AddComponent<Enemy>();
        enemy2.enemyName = "Orc";
        enemyList.AddItem(enemy2);

        Item healthPotion = new GameObject("Item").AddComponent<Item>();
        healthPotion.itemName = "Health Potion";
        itemList.AddItem(healthPotion);

        Item manaPotion = new GameObject("Item").AddComponent<Item>();
        manaPotion.itemName = "Maná Potion";
        itemList.AddItem(manaPotion);

        foreach (Item items in itemList.GetItems())
        {
            Debug.Log("Enemy in list: " + items.itemName);
        }

        foreach (Enemy enemy in enemyList.GetItems())
        {
            Debug.Log("Enemy in list: " + enemy.enemyName);
        }
    }
}
```

Creamos una clase para implementar dos listas distintas utilizando una clase que acepte parametros genéricos.

CASO DE USO

```
public class GameController : MonoBehaviour
{
    private ClaseGenerica<Enemy> enemyList;
    private ClaseGenerica<Item> itemList;

    void Start()
    {
        enemyList = new ClaseGenerica<Enemy>();
        itemList = new ClaseGenerica<Item>();
    }
}
```

- **private ClaseGenerica<Enemy> enemyList:** Declara un campo privado enemyList que es una instancia de ClaseGenerica que maneja objetos de tipo Enemy.
- **private ClaseGenerica<Item> itemList:** Declara un campo privado itemList que es una instancia de ClaseGenerica que maneja objetos de tipo Item.

CASO DE USO

```
public class GameController : MonoBehaviour
{
    private ClaseGenerica<Enemy> enemyList;
    private ClaseGenerica<Item> itemList;

    void Start()
    {
        enemyList = new ClaseGenerica<Enemy>();
        itemList = new ClaseGenerica<Item>();
    }
}
```

- **private ClaseGenerica<Enemy> enemyList:** Declara un campo privado enemyList que es una instancia de ClaseGenerica que maneja objetos de tipo Enemy.
- **private ClaseGenerica<Item> itemList:** Declara un campo privado itemList que es una instancia de ClaseGenerica que maneja objetos de tipo Item.

Se crean nuevas instancias de ClaseGenerica para manejar enemigos y objetos respectivamente.

CASO DE USO

```
Enemy enemy1 = new GameObject("Enemy1").AddComponent<Enemy>();
enemy1.enemyName = "Goblin";
enemyList.AddItem(enemy1);

Enemy enemy2 = new GameObject("Enemy2").AddComponent<Enemy>();
enemy2.enemyName = "Orc";
enemyList.AddItem(enemy2);
```

- Se crean nuevos GameObject llamados "Enemy1" y "Enemy2".
- Se añaden componentes Enemy a estos objetos.
- Se asignan nombres a los enemigos ("Goblin" y "Orc").
- Se añaden estos enemigos a la enemyList.

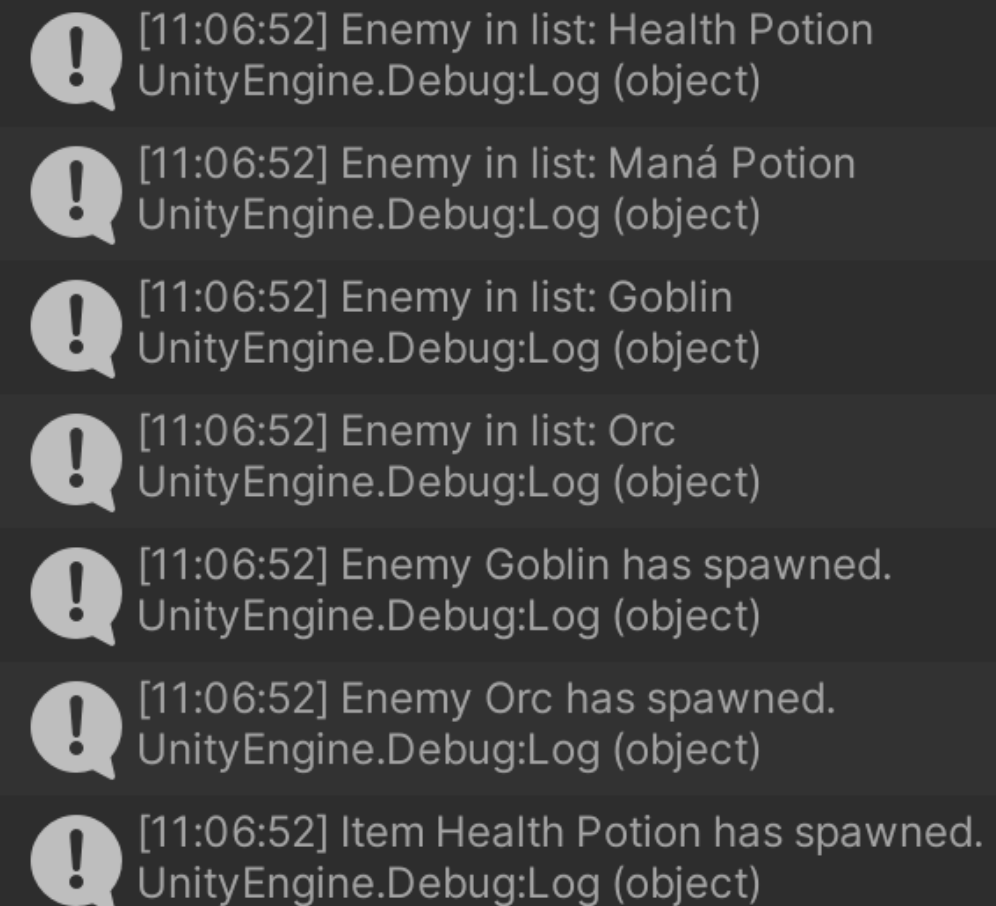
- Se crean nuevos GameObject llamados "Item".
- Se añaden componentes Item a estos objetos.
- Se asignan nombres a los objetos ("Health Potion" y "Maná Potion").
- Se añaden estos objetos a la itemList.

```
Item healthPotion = new GameObject("Item").AddComponent<Item>();
healthPotion.itemName = "Health Potion";
itemList.AddItem(healthPotion);

Item manaPotion = new GameObject("Item").AddComponent<Item>();
manaPotion.itemName = "Maná Potion";
itemList.AddItem(manaPotion);
```

CASO DE USO

```
foreach (Item items in itemList.GetItem())  
{  
    Debug.Log("Enemy in list: " + items.itemName);  
}  
  
foreach (Enemy enemy in enemyList.GetItem())  
{  
    Debug.Log("Enemy in list: " + enemy.enemyName);  
}
```



The screenshot shows a Unity console with seven log entries, each preceded by a yellow warning icon. The logs are as follows:

- [11:06:52] Enemy in list: Health Potion
UnityEngine.Debug:Log (object)
- [11:06:52] Enemy in list: Maná Potion
UnityEngine.Debug:Log (object)
- [11:06:52] Enemy in list: Goblin
UnityEngine.Debug:Log (object)
- [11:06:52] Enemy in list: Orc
UnityEngine.Debug:Log (object)
- [11:06:52] Enemy Goblin has spawned.
UnityEngine.Debug:Log (object)
- [11:06:52] Enemy Orc has spawned.
UnityEngine.Debug:Log (object)
- [11:06:52] Item Health Potion has spawned.
UnityEngine.Debug:Log (object)

- Se iteran todos los objetos en itemList y se imprimen sus nombres usando Debug.Log.
- Se iteran todos los enemigos en enemyList y se imprimen sus nombres usando Debug.Log.

CONCLUSIÓN

Como podemos ver en estos ejemplos, el uso de genéricos en Unity nos permite crear código más flexible, limpio y reutilizable. Los genéricos nos ayudan a evitar la duplicación de código y a mantener nuestro proyecto más organizado y fácil de mantener.